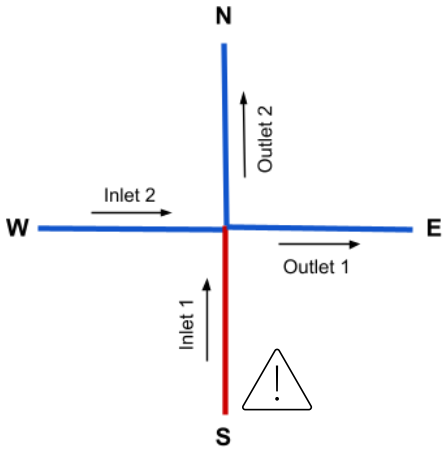
EPANET – IMX

EPANET – IMX est l'application EPANET modifié dans lequel un mélange incomplet se produit dans les jonctions en croix au lieu d'un mélange complet comme dans l'application original.

EPANET – IMX is the modified EPANET application in which incomplete mixing occurs in cross junctions instead of complete mixing as in the original application.

La concentration des jonctions sortantes sont calculés selon le modèle de mélange incomplet de Reza Yousefian :

The concentration of outlet junctions is calculated according to Reza Yousefian's incomplete mixing model:



$$Q_{Outlet1}/Q_{Inlet1} \leq 0.85 \Rightarrow C^*_{Outlet1} = 1.00$$

$$Q_{Outlet1}/Q_{Inlet1} > 0.85 \Rightarrow C^*_{Outlet1} = 0.022 \times \ln\left(\frac{Q_{Outlet1}}{Q_{Inlet2}}\right) + 0.91 \times \left(\frac{Q_{Outlet1}}{Q_{Inlet1}}\right)^{-0.79}$$

Source: Improving incomplete mixing modeling for junctions of water distribution networks

Où trouver le code source IMX *Where to find IMX source code*

Vous pouvez trouver le code source de EPANET-IMX ici *You can find the source code for EPANET-IMX here* : EPANET2.3-IMX

Pour installer le répertoire EPANET-IMX sur votre ordinateur, vous pouvez simplement faire « Download Zip » en appuyant sur le bouton vert « Code ». Ou, si vous êtes à l'aise avec GitHub, vous pouvez simplement faire un git clone de l'URL dans le terminal de votre ordinateur (cmd) ou celui de votre IDE. Vous pouvez déplacer le dossier du code décompressé où vous voulez sur votre ordinateur, tant que c'est à un endroit accessible où vous ne l'oublierez pas.

To install the EPANET-IMX directory on your computer, you can simply click "Download Zip" by pressing the green "Code" button. Or, if you are comfortable with GitHub, you can simply create a ` git

clone` of the URL in your computer's terminal (cmd) or your IDE's terminal. You can move the extracted code folder anywhere on your computer, as long as it's in an easily accessible location where you won't forget about it.

Pour simplifier les étapes à la fin du guide, changer le nom du dossier « EPANET2.3-IMX-main » à « EPANET2.3-IMX »

To simplify the steps at the end of the guide, change the folder name from "EPANET2.3-IMX-main" to "EPANET2.3-IMX".

Explication des modifications de code *Code modifications explanation*

Par souci de ne pas prendre trop d'espace, l'explication du code se fera en anglais, puisque l'article sur lequel on se base et le langage de programmation sont en anglais. Si vous ne comprenez pas, je vous invite à traduire les explications avec Google Traduction.

To avoid taking up too much space, the code explanation will be in English.

In types.h :

Line 409:
Before the functions, there's a "structure". Think of it as a record that groups together several pieces of information about a single junction (a pipe crossing). Each junction has its own record containing: its number, whether it's a cross junction, which pipe brings in the contaminated water, which pipe brings in the pure water, etc.

```
typedef struct
{
  int nodeindex;     The junction's number
  int iscrossjunc;   1 if it's a cross junction, 0 otherwise
  int contaminlink;  The inflow pipe carrying the more concentrated water
  int purelink;      The inflow pipe carrying the "cleaner" (less concentrated) water
  int adjoutlink;    The "adjacent" outflow pipe (adjacent to the contaminated pipe)
  int oppoutlink;    The "opposite" outflow pipe (parallel/opposite to the contaminated pipe)
  double contaminconc;       The concentration of the contaminated water
  double pureconc;   The concentration of the pure water
} Cjunc;
```

In quality.c :

Line 719 :
This function loops through every node (junction) in the network and figures out which ones are cross junctions, meaning spots where 2 pipes bring water in and 2 other pipes carry it out, with roughly a 90° angle between the pipes. This matters because at this kind of layout, water doesn't mix perfectly, unlike what EPANET normally assumes.

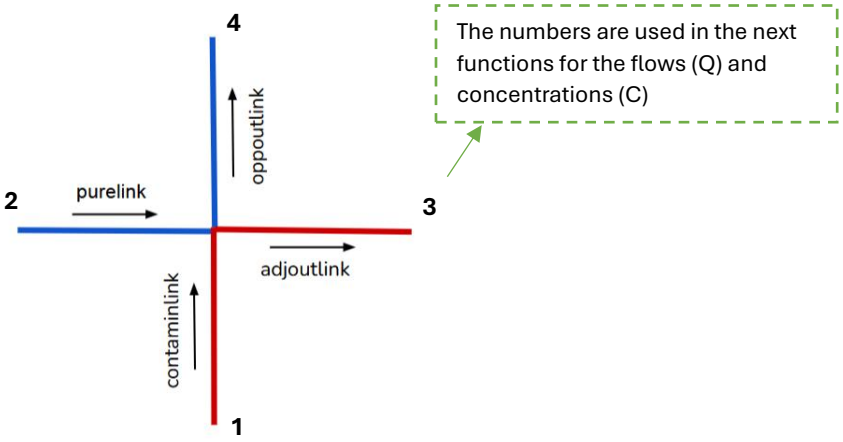
```
int findcrossjuncs(Project *pr)
{
    Network *net = &pr->network;
```

```
Hydraul *hyd = &pr->hydraul;
Quality *qual = &pr->quality;

int j, k;
int Nupnode, Ndownnode;
int inlink1, inlink2, outlink1, outlink2;
int opplink1, opplink2;

qual->Ncrossjuncs = 0;

Go through every junction in the network, one at a time
for (j = 1; j <= net->Njuncs; j++)
{
    qual->Crossjuncs[j].iscrossjunc = 0;

    If the junction has no coordinates (X, Y) in the file, skip it (angles can't be calculated without
    knowing its physical location)
    if (net->Node[j].X == MISSING || net->Node[j].Y == MISSING) continue;

    Nupnode = 0;        Counts the number of pipes flowing OUT of this junction
    Ndownnode = 0;      Counts the number of pipes flowing INTO this junction

    Go through every pipe in the network to see if it's connected to junction j, and in which direction
    water is flowing through it
    for (k = 1; k <= net->Nlinks; k++)
    {
        if (net->Link[k].Len == 0.0) continue;    Skip zero-length pipes (ex. valves)

        Based on the flow direction (positive or negative) and whether the pipe starts (N1) or ends
        (N2) at this junction, determine whether it's an "inflow" or "outflow" pipe relative to the
        junction
        if (hyd->LinkFlow[k] > 0.0 && net->Link[k].N1 == j)
        {
            Nupnode++;
            if (Nupnode == 1) outlink1 = k;
            else if (Nupnode == 2) outlink2 = k;
        }
        else if (hyd->LinkFlow[k] > 0.0 && net->Link[k].N2 == j)
        {
            Ndownnode++;
            if (Ndownnode == 1) inlink1 = k;
            else if (Ndownnode == 2) inlink2 = k;
        }
        else if (hyd->LinkFlow[k] < 0.0 && net->Link[k].N2 == j)
        {
            Nupnode++;
            if (Nupnode == 1) outlink1 = k;
            else if (Nupnode == 2) outlink2 = k;
        }
        else if (hyd->LinkFlow[k] < 0.0 && net->Link[k].N1 == j)
        {
            Ndownnode++;
            if (Ndownnode == 1) inlink1 = k;
            else if (Ndownnode == 2) inlink2 = k;
```

```
        }
    }

    A junction is only considered a cross-junction if exactly 2 pipes flow in AND exactly 2 pipes flow out
    if (Nupnode == 2 && Ndownnode == 2)
    {
        This function identifies, among the pipes, which one is "across from" (opposite to) another,
        based on angles
        opplink1 = nonadjlink(pr, inlink1, inlink2, outlink1, outlink2, j);

        if (opplink1 != inlink2)
        {
            opplink2 = nonadjlink(pr, inlink2, inlink1, outlink1, outlink2, j);

            Calculate the angle between the two inflow pipes
            double a_in1 = getlinkangle(pr, inlink1, j);
            double a_in2 = getlinkangle(pr, inlink2, j);
            double crossAngle = fabs(a_in2 - a_in1);
            if (crossAngle > M_PI) crossAngle = 2.0 * M_PI - crossAngle;

            Allow a tolerance of 5 degrees around 90°
            double TOL = 5.0 * M_PI / 180.0;

            If the two inflow pipes don't form an angle close to 90°, this isn't a true cross junction,
            move on to the next junction
            if (fabs(crossAngle - M_PI / 2.0) > TOL) continue;

            Determine which incoming pipe belongs to the primary flow path. We compare the total
            absolute flow (inflow + opposite outflow) for both pipes. The pipe associated with the
            larger total flow is designated as the primary/contaminated link (cl).
            int cl = inlink1, pl = inlink2;
            if ((fabs(hyd->LinkFlow[inlink2]) + fabs(hyd->LinkFlow[opplink2])) >
                (fabs(hyd->LinkFlow[inlink1]) + fabs(hyd->LinkFlow[opplink1])))
            {
                cl = inlink2;
                pl = inlink1;
            }

            Determine which outflow pipe is "adjacent" (closest in angle) and which one is
            "opposite" relative to the contaminated pipe
            double a_cl      = getlinkangle(pr, cl, j);
            double diff_out1 = fabs(fmod(fabs(getlinkangle(pr, outlink1, j) -
a_cl), 2.0*M_PI) - M_PI);
            double diff_out2 = fabs(fmod(fabs(getlinkangle(pr, outlink2, j) -
a_cl), 2.0*M_PI) - M_PI);

            int opp_out = (diff_out1 < diff_out2) ? outlink1 : outlink2;
            int adj_out = (diff_out1 < diff_out2) ? outlink2 : outlink1;

            Save all this information onto the junction's “record”
            qual->Ncrossjuncs++;
            qual->Crossjuncs[j].iscrossjunc  = 1;
            qual->Crossjuncs[j].nodeindex    = j;
            qual->Crossjuncs[j].contaminlink = cl;
```

```
                qual->Crossjuncs[j].purelink     = pl;
                qual->Crossjuncs[j].oppoutlink   = opp_out;
                qual->Crossjuncs[j].adjoutlink   = adj_out;
            }
        }
    }
    return 0;
}
```

Diagram of the pipe names in the code:



Line 843 :

Once a junction is known to be a cross-junction, this function calculates what the water concentration will be in the closest (by angle) outflow pipe to the most contaminated inflow pipe. Water doesn't fully mix at a cross junction, so the outflow pipe “adjacent” to the contaminated pipe will keep a concentration closer to it, based on a flow ratio.

```
double imxadjoutconc(Project *pr, Cjunc *cj)
{
    Hydraul *hyd = &pr->hydraul;

    Get the flows (as absolute values, so direction doesn't matter for the three pipes involved
    double Q1 = fabs(hyd->LinkFlow[cj->contaminlink]);       Flow in the contaminated pipe
    double Q2 = fabs(hyd->LinkFlow[cj->purelink]);    Flow in the "pure" pipe
    double Q3 = fabs(hyd->LinkFlow[cj->adjoutlink]); Flow in the adjacent outflow pipe
    double Ccont = cj->contaminconc;     Concentration of the contaminated water
    double Cpur  = cj->pureconc; Concentration of the pure water
    double ratio = Q3 / Q1;       Proportion of the outflow relative to the contaminated inflow

    If the two concentrations are essentially identical, no need to calculate: the result is simply that
    concentration
    if (fabs(Ccont - Cpur) < 1e-10) return Ccont;

    C3_star represents a fraction (between 0 and 1) indicating "how much" the outgoing water resembles
    the contaminated water versus the pure water. C3_star is the dimensionless concentration of the
    adjacent outlet. If the flow ratio is small (<= 0.85), it's assumed to be nearly entirely made up of
    contaminated water (value 1.0).  Otherwise, the formula from the reference article is used
    double C3_star;
    if (ratio <= 0.85)
```

```
        C3_star = 1.0;
    else
        C3_star = 0.022 * log(Q3/Q2) + 0.91 * pow(ratio, -0.79);

    Make sure C3_star stays between 0 and 1 (values outside this range wouldn't make physical sense)
    if (C3_star < 0.0) C3_star = 0.0;
    if (C3_star > 1.0) C3_star = 1.0;

    Calculate the final concentration (in mg/L or other units): a weighted mix between the contaminated and
    pure water, based on C3_star (fraction) calculated above
    double result = C3_star * (Ccont - Cpur) + Cpur;

    Safety check. The final result can never be "cleaner" than Cpur, nor more "contaminated" than Ccont
    if (result < Cpur)  result = Cpur;
    if (result > Ccont) result = Ccont;

    return result;
}
```

Line 894:

This function calculates the concentration in the other outflow pipe (the one "across from" (opposite) the first one, not adjacent). Rather than using an empirical formula like the previous function and in the reference article, it relies on a mass balance: what goes in must equal what comes out, so once the adjacent outlet's concentration is known, the opposite outlet's concentration can be derived by calculation.

```
double imxoppoutconc(Project *pr, Cjunc *cj)
{
    Hydraul *hyd = &pr->hydraul;

    double Q1 = fabs(hyd->LinkFlow[cj->contaminlink]);      contaminated inflow
    double Q2 = fabs(hyd->LinkFlow[cj->purelink]);   pure inflow
    double Q3 = fabs(hyd->LinkFlow[cj->adjoutlink]); adjacent outflow
    double Q4 = fabs(hyd->LinkFlow[cj->oppoutlink]); opposite outflow
    double Ccont = cj->contaminconc;
    double Cpur  = cj->pureconc;

    Same safety checks as in the previous function: if the concentrations are identical, or if the opposite
    outflow is nearly zero, no need to calculate
    if (fabs(Ccont - Cpur) < 1e-10) return Ccont;
    if (Q4 < 1e-10) return Cpur;

    Reuse the previous function to get the concentration of the adjacent outflow pipe
    double C3 = imxadjoutconc(pr, cj);

    Mass balance: the amount of contaminant coming in (Q1*Ccont + Q2*Cpur) must equal the amount
    going out (Q3*C3 + Q4*C4). So C4 (the value being calculated) is isolated from this equation.
    double result = (Ccont * Q1 + Cpur * Q2 - C3 * Q3) / Q4;

    Again, make sure the result stays within a realistic range
    if (result < Cpur)  result = Cpur;
    if (result > Ccont) result = Ccont;
```

```
    return result;
}
```

Line 937:
A basic math function that calculates the angle (in radians) between two points. It's used to determine which direction each pipe points in, starting from a junction.

```
double angle(double x1, double y1, double x2, double y2)
{
    double relptx = x1 - x2;
    double relpty = y1 - y2;
    double arctanpt = atan(relpty / relptx);

    Depending on which quadrant (up/down, left/right) the point falls in, the angle is adjusted so it's always
    expressed between 0 and 360° (2π radians)
    if ((relptx >= 0.0) && (relpty >= 0.0)) return arctanpt;
    else if ((relptx <= 0.0) && (relpty >= 0.0)) return arctanpt + M_PI;
    else if ((relptx < 0.0) && (relpty <= 0.0)) return arctanpt + M_PI;
    else if ((relptx >= 0.0) && (relpty <= 0.0)) return arctanpt + 2 * M_PI;
    else return 0.0;
}
```

Line 962:
To calculate a pipe's angle, its coordinates need to be known first. This function looks up the pipe's coordinate closest to the junction, either an intermediate point (a point along the pipe's path on the map), or, if there isn't one, simply the coordinates of the pipe's other end.

```
void getlinkcoords(Project *pr, int lnk, int node, double *x, double *y)
{
    Network *net = &pr->network;
    Slink *link = &net->Link[lnk];
    Pvertices verts = link->Vertices; The intermediate points along the pipe's path (if any)

    If the pipe has intermediate points (it's not just a straight line), use the first intermediate point closest to
    the junction
    if (verts != NULL && verts->Npts > 0)
    {
        if (link->N1 == node)
        {
            *x = verts->X[0];
            *y = verts->Y[0];
        }
        else
        {
            *x = verts->X[verts->Npts-1];
            *y = verts->Y[verts->Npts-1];
        }
    }

    Otherwise (a straight pipe, with no intermediate point), simply use the coordinates of the pipe's other
    end
    else
    {
```

```
        int other = (link->N1 == node) ? link->N2 : link->N1;
        *x = net->Node[other].X;
        *y = net->Node[other].Y;
    }
}
```

Line 1005:
Determines the geometrically opposite pipe among three candidate links relative to a reference pipe. This is what allows findcrossjuncs to tell the adjacent pipe apart from the opposite one.

```
int nonadjlink(Project *pr, int tolnk, int lnk2, int lnk3, int lnk4, int node)
{
    Calculate the angle of each pipe relative to the junction
    double a1 = getlinkangle(pr, tolnk, node);
    double a2 = getlinkangle(pr, lnk2, node);
    double a3 = getlinkangle(pr, lnk3, node);
    double a4 = getlinkangle(pr, lnk4, node);

    Express the other pipes' angles relative to the reference pipe
    a2 -= a1;
    a3 -= a1;
    a4 -= a1;

    Bring these angles back within the 0–360° range
    if (a2 < 0.0) a2 += (2.0 * M_PI);
    if (a3 < 0.0) a3 += (2.0 * M_PI);
    if (a4 < 0.0) a4 += (2.0 * M_PI);

    Check which of the three pipes falls "between" the other two in terms of angle: that one is considered
    the opposite pipe
    if (((a2 >= a3) && (a2 <= a4)) || ((a2 <= a3) && (a2 >= a4))) return lnk2;
    if (((a3 >= a2) && (a3 <= a4)) || ((a3 <= a2) && (a3 >= a4))) return lnk3;
    if (((a4 >= a2) && (a4 <= a3)) || ((a4 <= a2) && (a4 >= a3))) return lnk4;
    return lnk2;
}
```

Line 1046:
This function combines the two previous ones (getlinkcoords and angle): it first finds the coordinates of a pipe near the junction, then calculates the angle between that point and the junction itself. It's a "shortcut" function used throughout the rest of the code.

```
double getlinkangle(Project *pr, int lnk, int node)
{
    double x, y;
    getlinkcoords(pr, lnk, node, &x, &y); find the pipe's coordinates
    return angle(x, y, pr->network.Node[node].X, pr->network.Node[node].Y); calculate
    the angle
}
```

Line 1064:

After identifying a cross junction, this function checks that "contaminated" was correctly assigned to the pipe with the higher concentration, and "pure" to the other one. If that's not the case, it swaps the two and adjusts the adjacent/opposite outflow pipes accordingly.

```
void assigncontaminationnode(Project *pr, int n)
{
    Quality *qual = &pr->quality;
    Cjunc *cj = &qual->Crossjuncs[n];

    If this isn't a cross junction, do nothing
    if (!cj->iscrossjunc) return;

    If the pipe labeled "pure" actually has a higher concentration than the pipe labeled "contaminated," a
    mistake was made: swap them
    if (cj->pureconc > cj->contaminconc)
    {
        Swap the contaminated and pure pipes
        int tmplink      = cj->contaminlink;
        cj->contaminlink = cj->purelink;
        cj->purelink     = tmplink;

        Also swap their concentrations
        double tmpc      = cj->contaminconc;
        cj->contaminconc = cj->pureconc;
        cj->pureconc     = tmpc;

        Since which pipe is "contaminated" has changed, it's also necessary to check whether the
        "adjacent" outflow pipe is still the correct one (the one closest in angle to the new contaminated
        pipe). If not, swap those too
        int oldadj = cj->adjoutlink;
        int oldopp = cj->oppoutlink;
        double a_contam = getlinkangle(pr, cj->contaminlink, n);
        double a_adj    = getlinkangle(pr, oldadj, n);
        double a_opp    = getlinkangle(pr, oldopp, n);
        double diff_adj = fabs(fmod(fabs(a_adj - a_contam), 2*M_PI) - M_PI);
        double diff_opp = fabs(fmod(fabs(a_opp - a_contam), 2*M_PI) - M_PI);

        if (diff_adj < diff_opp)
        {
            cj->oppoutlink = oldadj;
            cj->adjoutlink = oldopp;
        }
    }
}
```

In qualroute.c :

Line 64:
This function already existed in EPANET (it manages the transport of contamination from one node to another at each time step), but additions have been made to account for cross junctions. Specifically, at each node, in addition to the normal calculation, it now checks whether a junction is a cross junction and, if so, special formulas (imxadjoutconc and

imxoppoutconc, seen previously) are used instead of the standard calculation to determine the concentration at the outlet.

```
void transport(Project *pr, long tstep)
{
    Network *net = &pr->network;
    Hydraul *hyd = &pr->hydraul;
    Quality *qual = &pr->quality;

    int j, k, m, n;
    int cj_err;
    double volin, massin, volout, nodequal;
    Padjlist alink;

    Calculation of chemical reactions in pipes and tanks (This already existed, no changes)
    if (qual->Reactflag)
    {
        reactpipes(pr, tstep);
        reacttanks(pr, tstep);
    }

    NEW: Before starting the calculation, we identify all the cross junctions of the network (see the
    findcrossjuncs function seen previously)
    cj_err = findcrossjuncs(pr);
    if (cj_err > 0) return;

    We process each node of the network, one at a time, in a specific order (already determined elsewhere
    and stored in SortedNodes)
    for (j = 1; j <= net->Nnodes; j++)
    {
        n = qual->SortedNodes[j];

        volin  = 0.0; Volume of water entering this node
        massin = 0.0; Mass of contaminant entering this node
        volout = 0.0; Volume of water leaving this node

        We examine all the pipes connected to this node to calculate how much water (and contaminant)
        enters and exits it
        for (alink = net->Adjlist[n]; alink != NULL; alink = alink->next)
        {
            k = alink->link;

            m = (qual->FlowDir[k] < 0) ? net->Link[k].N1 : net->Link[k].N2;
            if (m == n)
            {
                This pipe brings water to our node: we calculate how much volume and contaminant it
                brings
                evalnodeinflow(pr, k, tstep, &volin, &massin);
            }

            else volout += fabs(LINKFLOW(k)); Otherwise, this pipe drains water
        }

        If there is a demand for water at this node, it is added to the outgoing volume.
```

```
if (net->Node[n].Type == JUNCTION)
{
    volout += MAX(0.0, hyd->NodeDemand[n]);
}

volout *= tstep;  Convert the flow rate into volume over the duration of the time step

NEW: If this node is a cross junction, we will find the concentration of each of the two incoming
pipes (the contaminated one and the clean one), in order to then perform the special calculations
if (n <= net->Njuncs && qual->Crossjuncs[n].iscrossjunc == 1)
{
    Cjunc *cj = &qual->Crossjuncs[n];
    int inlinks[2] = { cj->contaminlink, cj->purelink };
    double concs[2];

    for (int ii = 0; ii < 2; ii++)
    {
        k = inlinks[ii];

        If the pipe has a water quality "segment" at its outlet, we take its concentration directly
        if (qual->LastSeg[k] != NULL)
            concs[ii] = qual->LastSeg[k]->c;
        else
        {
            Or else, we take the concentration of the node upstream of the pipe
            int upnode = (qual->FlowDir[k] >= 0)
                        ? net->Link[k].N1
                        : net->Link[k].N2;
            concs[ii] = qual->NodeQual[upnode];
        }
    }
    cj->contaminconc = concs[0];
    cj->pureconc     = concs[1];

    We check (and correct if necessary) that the "contaminated" pipe is indeed the one with the
    highest concentration (see the function assigncontaminationnode seen previously)
    assigncontaminationnode(pr, n);
}

Standard calculation of concentration at the node (already existing in EPANET)
nodequal = findnodequal(pr, n, volin, massin, volout, tstep);

This concentration is now distributed to the outgoing pipes
for (alink = net->Adjlist[n]; alink != NULL; alink = alink->next)
{
    k = alink->link;
    m = (qual->FlowDir[k] < 0) ? net->Link[k].N2 : net->Link[k].N1;
    if (m == n)
    {
        double outconc = nodequal;  By default, we use the normally calculated
                                    concentration

        NEW: If this node is a cross junction, we replace this standard calculation with special
        formulas, depending on whether the outgoing pipe is the "adjacent" or "opposite" pipe
```

```
                if (n <= net->Njuncs && qual->Crossjuncs[n].iscrossjunc == 1)
                {
                    Cjunc *cj = &qual->Crossjuncs[n];
                    if (k == cj->adjoutlink)
                        outconc = imxadjoutconc(pr, cj);
                    else if (k == cj->oppoutlink)
                        outconc = imxoppoutconc(pr, cj);
                }
                This concentration is applied to the outgoing pipe
                evalnodeoutflow(pr, k, outconc, tstep);
            }
        }
        We update the overall mass balance (contaminant accounting total in the network, already
        existing)
        updatemassbalance(pr, n, massin, volout, tstep);
    }
}
```

Line 266:

This function already existed and calculates the standard concentration of a node. The only addition here is a small early exit: if the node is a cross junction, the function stops earlier and directly returns the base concentration, without going through the additional steps normally required for contamination sources or the "trace" mode. This is because, for a cross junction, the final concentration is usually calculated by the special functions mentioned earlier (imxadjoutconc/imxoppoutconc), not here.

```
double findnodequal(Project *pr, int n, double volin,
                    double massin, double volout, long tstep)
{
    Network *net = &pr->network;
    Hydraul *hyd = &pr->hydraul;
    Quality *qual = &pr->quality;

    if (net->Node[n].Type == JUNCTION)
    {
        We remove the portion of the incoming volume that actually corresponds to a negative demand (ex.
        an injection of clean water at this node)
        volin -= MIN(0.0, hyd->NodeDemand[n]) * tstep;

        Basic calculation: concentration = incoming mass / incoming volume
        if (volin > 0.0) qual->NodeQual[n] = massin / volin;

        If there is no incoming volume, the calculation is done differently (function already exists for cases
        without flow)
        else if (qual->Reactflag) qual->NodeQual[n] = noflowqual(pr, n);

        NEW: If it's a cross junction, we stop here and directly return this base concentration. The
        calculations special for cross junctions will be done later, in the transport() function seen above.
        if (n <= net->Njuncs && qual->Crossjuncs[n].iscrossjunc == 1)
            return qual->NodeQual[n];
    }

    If it's a tank, the mixture is calculated differently (existing function)
```

```
    else if (net->Node[n].Type == TANK)
    {
        qual->NodeQual[n] = mixtank(pr, n, volin, massin, volout);
    }

    qual->SourceQual = 0.0;

    Special case of "trace" mode (tracing a precise source), already existing, not related to cross junctions
    if (qual->Qualflag == TRACE)
    {
        if (n == qual->TraceNode)
        {
            if (net->Node[n].Type == RESERVOIR) qual->SourceQual = 100.0;
            else qual->SourceQual = MAX(100.0 - qual->NodeQual[n], 0.0);
            qual->NodeQual[n] = 100.0;
        }
        return qual->NodeQual[n];
    }

    Adding a potential external source of contamination to this node (already existing)
    qual->SourceQual = findsourcequal(pr, n, volout, tstep);
    if (qual->SourceQual == 0.0) return qual->NodeQual[n];

    switch (net->Node[n].Type)
    {
        case JUNCTION:
            qual->NodeQual[n] += qual->SourceQual;
            return qual->NodeQual[n];
        case TANK:
            return qual->NodeQual[n] + qual->SourceQual;
        case RESERVOIR:
            qual->NodeQual[n] = qual->SourceQual;
            return qual->SourceQual;
    }
    return qual->NodeQual[n];
}
```

Compiler le code en .dll *Compile the code in .dll*

Le .dll (Dynamic Link Library) contient le moteur de calcul d'EPANET, soit le code qui effectue les simulations hydrauliques.

Après avoir effectué des changements dans le code source, il faut compiler le code source en .dll et le GUI en .exe pour pouvoir effectuer des essais ou des simulations de réseaux dans EPANET-IMX.

La première étape s'agit de compiler le code source en un .dll.

The .dll (Dynamic Link Library) contains the EPANET calculation engine, which is the code that performs the hydraulic simulations.

After making changes to the source code, the source code must be compiled into a .dll file and the GUI into an .exe file to perform network tests or simulations in EPANET-IMX.

The first step is to compile the source code into a .dll file.

1. Installer MSYS2 *Install MSYS2*

MSYS2 est une plateforme de distribution et de compilation de logiciels pour Windows offrant un environnement de type Unix. Elle associe le gestionnaire de paquets Pacman aux chaînes d'outils MinGW-w64 pour compiler des logiciels Windows natifs à l'aide d'outils tels que GCC, Bash et Make. C'est cette plateforme que nous allons utiliser pour compiler le code source de EPANET-IMX en un .dll. Notez qu'elle fonctionne seulement sur Windows (pas sur Mac ni Linux).

Pour l'installer, suivre les étapes 1 à 5 du site suivant :

MSYS2 is a software distribution and compilation platform for Windows that provides a Unix-like environment. It combines the Pacman package manager with the MinGW-w64 toolchain to compile native Windows software using tools such as GCC, Bash, and Make. This is the platform we will use to compile the EPANET-IMX source code into a .dll file. Note that it only works on Windows (not on Mac or Linux).

To install it, follow the steps 1 to 5 on the following website:

MSYS2 - Software Distribution and Building Platform for Windows

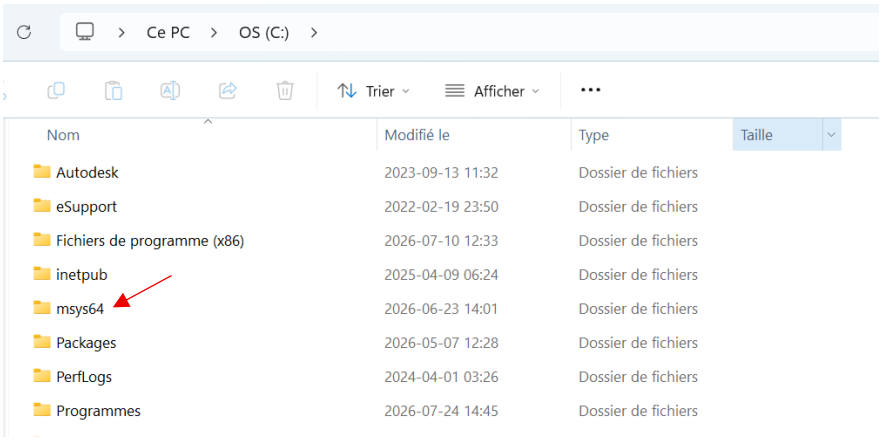
2. Ouvrir mingw32 et installer les outils *Open mingw32 and install the tools*

Puisque EPANET a été développé en 32 bits, nous ne pouvons malheureusement pas compiler le .dll en 64 bits, car cela ne fonctionnera pas et engendrera des erreurs. Pour compiler le .dll en 32 bits, il faudra utiliser mingw32.

Vous retrouverez mingw32 dans le dossier msys64 qui a été installé sur votre disque C : (ou autre chemin que vous aviez spécifié à l'installation) :

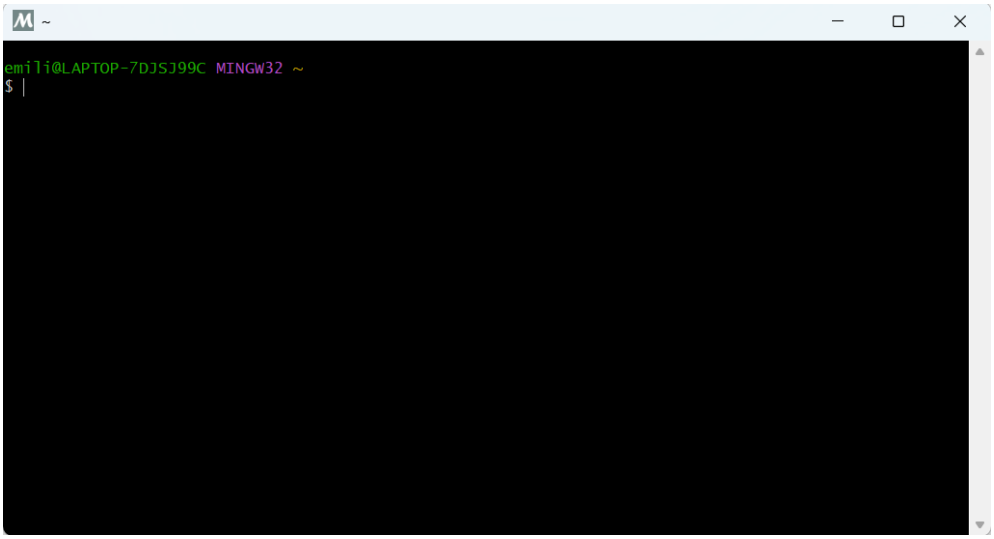
Since EPANET was developed in 32-bit, we unfortunately cannot compile the .dll in 64-bit, as this will not work and will generate errors. To compile the .dll in 32-bit, you will need to use mingw32.

You will find mingw32 in the msys64 folder that was installed on your C: drive (or the other path you specified during installation):



Par la suite, descendez dans le dossier et trouvez l'application mingw32, puis ouvrez-la (double-clic ou ouvrir par clic droit). Un terminal devrait s'ouvrir :

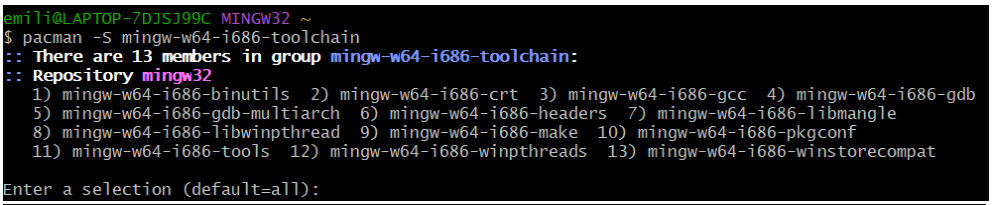
Next, navigate down to the folder and find the mingw32 application, then open it (double-click or right-click). A terminal should open:

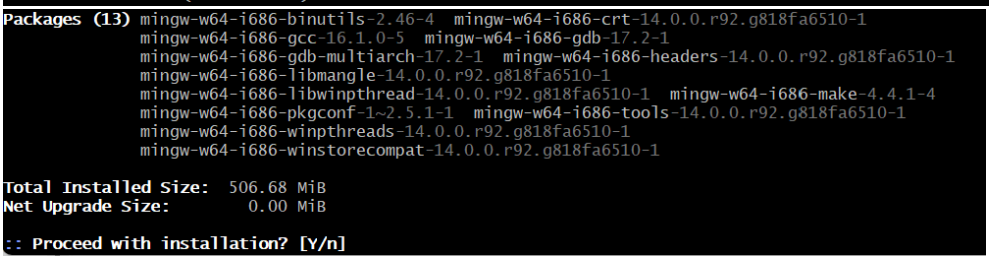


Dans le terminal, vous allez installer la chaîne d'outils GCC 32 bits. Copiez-collez la ligne suivante (Ctrl-C, Ctrl-V ne fonctionne pas dans le terminal, il faudra copier-coller par clic droit) et appuyez sur Enter. Le terminal affichera « Enter a selection (default = all) » : appuyez sur Enter. Il affichera ensuite « Proceed with installation? [Y/n] » : appuyez sur Enter encore pour tout installer (ou tapez Y et Enter).

In the terminal, you will install the 32-bit GCC toolchain. Copy and paste the following line (Ctrl-C, Ctrl-V do not work in the terminal; you will need to copy and paste by right-clicking) and press Enter. The terminal will display "Enter a selection (default = all)": press Enter. It will then display "Proceed with installation? [Y/n]": press Enter again to install everything (or type Y and press Enter).

pacman -S mingw-w64-i686-toolchain

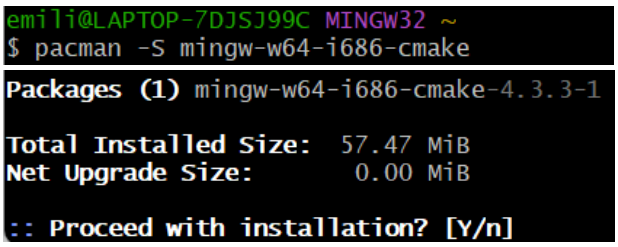




Ensuite, vous allez installer Cmake (version native 32-bits). Copiez-collez la ligne suivante et appuyez sur Enter. Le terminal affichera « Proceed with installation? [Y/n] » et appuyer encore sur Enter.

Next, you will install CMake (native 32-bit version). Copy and paste the following line and press Enter. The terminal will display "Proceed with installation? [Y/n]"; press Enter again.

pacman -S mingw-w64-i686-cmake



3. Compiler le code source EPANET-IMX en un .dll *Compile the EPANET-IMX source code into a .dll*

Dans le terminal mingw32, déplacez-vous dans le dossier EPANET2.3-IMX que vous avez installé, avec la commande suivante (le chemin change selon l'endroit où vous avez mis le dossier). Le chemin devrait être similaire à celui-ci :

In the mingw32 terminal, navigate to the EPANET2.3-IMX folder you installed using the following command (the path will vary depending on where you placed the folder). The path should be similar to this:

cd /c/Users/compte/Desktop/.../EPANET2.3-IMX

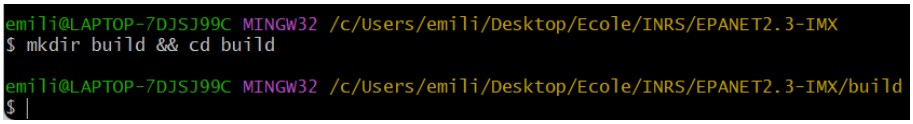
« compte » est le nom ou la version courte du nom de l'administrateur - par exemple, pour moi c'est « emili ». « Desktop » peut être remplacé par Downloads, Documents, OneDrive, etc., selon l'endroit où vous avez mis le dossier. Continuez ensuite le chemin jusqu'à ce que vous ayez EPANET2.3-IMX à la fin de la ligne.

Une fois dans le répertoire de EPANET2.3, copiez-collez la commande suivante pour créer un dossier build et vous y déplacer :

"compte" is the name or shortened version of the administrator's name - for example, mine is "emili". "Desktop" can be replaced with Downloads, Documents, OneDrive, etc., depending on where you placed the folder. Continue navigating the path until you see EPANET2.3-IMX at the end of the line.

Once in the EPANET2.3 directory, copy and paste the following command to create a build folder and navigate to it:

mkdir build && cd build



Ensuite, copiez-collez cette ligne et appuyez sur Enter pour compiler :

Next, copy and paste this line and press Enter to compile:

cmake -G "MinGW Makefiles" -DCMAKE_C_FLAGS="-static -static-libgcc -Depanet2_EXPORTS=1" -DCMAKE_SHARED_LINKER_FLAGS="-static -static-libgcc -Wl,--add-stdcall-alias" ..

```
emili@LAPTOP-7DJSJ99C MINGW32 /c/Users/emili/Desktop/Ecole/INRS/EPANET2.3-IMX/build
$ cmake -G "MinGW Makefiles"   -DCMAKE_C_FLAGS="-static -static-libgcc -Depanet2_EXPORTS=1"
 -DCMAKE_SHARED_LINKER_FLAGS="-static -static-libgcc -Wl,--add-stdcall-alias"   ..
CMake Deprecation Warning at CMakeLists.txt:28 (cmake_minimum_required):
  Compatibility with CMake < 3.10 will be removed from a future version of
  CMake.

  Update the VERSION argument <min> value.  Or, use the <min>...<max> syntax
  to tell CMake that the project requires at least <min> but has been updated
  to work with policies introduced by <max> or earlier.


-- The C compiler identification is GNU 16.1.0
-- The CXX compiler identification is GNU 16.1.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: C:/msys64/mingw32/bin/cc.exe - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: C:/msys64/mingw32/bin/c++.exe - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
CMake Deprecation Warning at run/CMakeLists.txt:2 (cmake_minimum_required):
  Compatibility with CMake < 3.10 will be removed from a future version of
  CMake.

  Update the VERSION argument <min> value.  Or, use the <min>...<max> syntax
  to tell CMake that the project requires at least <min> but has been updated
  to work with policies introduced by <max> or earlier.


-- Performing Test COMPILER_HAS_DEPRECATED_ATTR
-- Performing Test COMPILER_HAS_DEPRECATED_ATTR - Success
-- Configuring done (5.6s)
-- Generating done (0.2s)
-- Build files have been written to: C:/Users/emili/Desktop/Ecole/INRS/EPANET2.3-IMX/build
```

Finalement, vous pouvez terminer la compilation en exécutant la commande suivante :

Finally, you can complete the compilation by running the following command:

mingw32-make

```
-- Performing Test COMPILER_HAS_DEPRECATED_ATTR
[  2%] Building C object CMakeFiles/epanet2.dir/src/epanet.c.obj
[  5%] Building C object CMakeFiles/epanet2.dir/src/epanet2.c.obj
[  8%] Building C object CMakeFiles/epanet2.dir/src/flowbalance.c.obj
[ 11%] Building C object CMakeFiles/epanet2.dir/src/genmmd.c.obj
[ 14%] Building C object CMakeFiles/epanet2.dir/src/hash.c.obj
[ 17%] Building C object CMakeFiles/epanet2.dir/src/hydcoeffs.c.obj
[ 20%] Building C object CMakeFiles/epanet2.dir/src/hydraul.c.obj
[ 22%] Building C object CMakeFiles/epanet2.dir/src/hydsolver.c.obj
[ 25%] Building C object CMakeFiles/epanet2.dir/src/hydstatus.c.obj
[ 28%] Building C object CMakeFiles/epanet2.dir/src/inpfile.c.obj
[ 31%] Building C object CMakeFiles/epanet2.dir/src/input1.c.obj
[ 34%] Building C object CMakeFiles/epanet2.dir/src/input2.c.obj
[ 37%] Building C object CMakeFiles/epanet2.dir/src/input3.c.obj
[ 40%] Building C object CMakeFiles/epanet2.dir/src/leakage.c.obj
[ 42%] Building C object CMakeFiles/epanet2.dir/src/mempool.c.obj
[ 45%] Building C object CMakeFiles/epanet2.dir/src/output.c.obj
[ 48%] Building C object CMakeFiles/epanet2.dir/src/project.c.obj
[ 51%] Building C object CMakeFiles/epanet2.dir/src/quality.c.obj
[ 54%] Building C object CMakeFiles/epanet2.dir/src/qualreact.c.obj
[ 57%] Building C object CMakeFiles/epanet2.dir/src/qualroute.c.obj
[ 60%] Building C object CMakeFiles/epanet2.dir/src/report.c.obj
[ 62%] Building C object CMakeFiles/epanet2.dir/src/rules.c.obj
[ 65%] Building C object CMakeFiles/epanet2.dir/src/smatrix.c.obj
[ 68%] Building C object CMakeFiles/epanet2.dir/src/util/cstr_helper.c.obj
[ 71%] Building C object CMakeFiles/epanet2.dir/src/util/errormanager.c.obj
[ 74%] Building C object CMakeFiles/epanet2.dir/src/util/filemanager.c.obj
[ 77%] Building C object CMakeFiles/epanet2.dir/src/validate.c.obj
[ 80%] Linking C shared library bin\libepanet2.dll
[ 80%] Built target epanet2
[ 82%] Building C object run/CMakeFiles/runepanet.dir/main.c.obj
[ 85%] Linking C executable ..\bin\runepanet.exe
[ 85%] Built target runepanet
[ 88%] Building C object src/outfile/CMakeFiles/epanet-output.dir/src/epanet_output.c.obj
[ 91%] Building C object src/outfile/CMakeFiles/epanet-output.dir/__/util/errormanager.c.obj
[ 94%] Building C object src/outfile/CMakeFiles/epanet-output.dir/__/util/filemanager.c.obj
[ 97%] Building C object src/outfile/CMakeFiles/epanet-output.dir/__/util/cstr_helper.c.obj
[100%] Linking C shared library ..\..\bin\libepanet-output.dll
[100%] Built target epanet-output
```

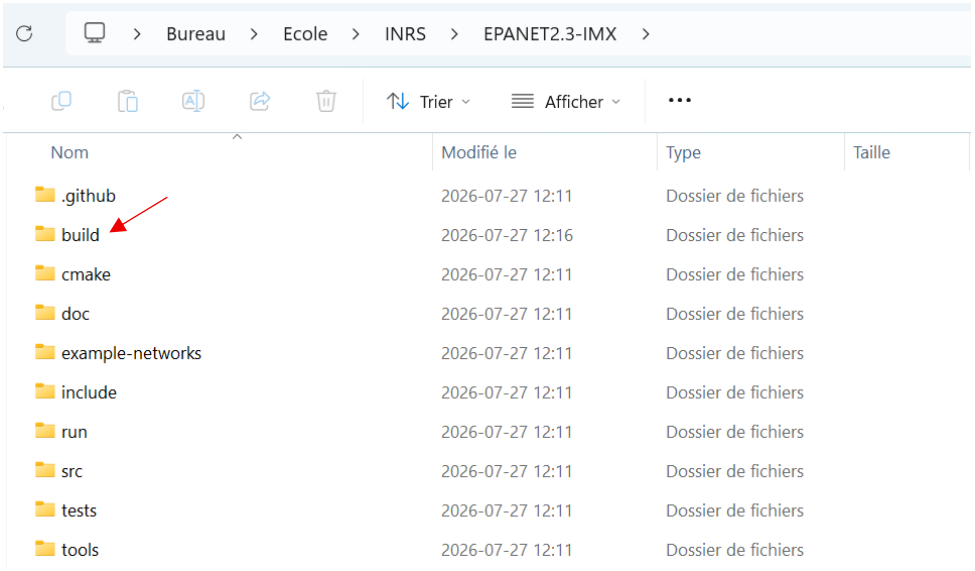
Vous pouvez fermer le terminal.

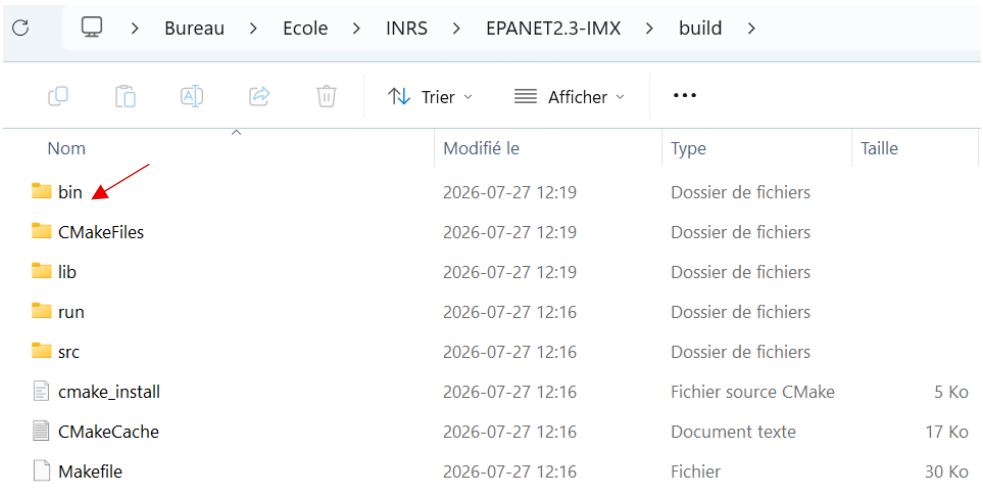
You can close the terminal.

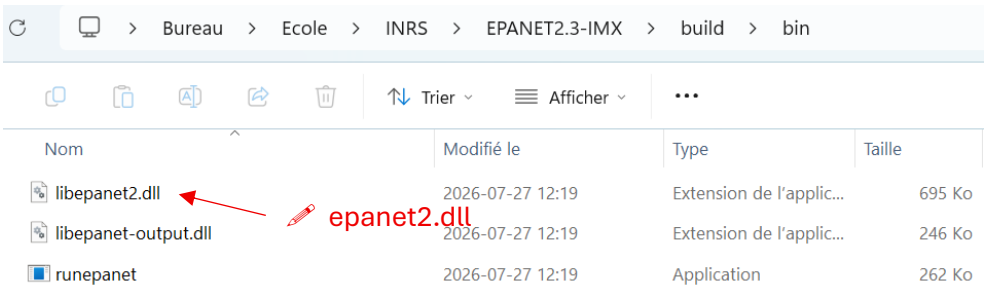
4. Changer le nom du .dll *Change the name of the .dll*

Pour la dernière étape, dirigez-vous dans le dossier EPANET2.3-IMX dans votre explorateur de fichiers. Allez ensuite dans le dossier « bin », vous y retrouverez un .dll nommé « libepanet2.dll ». Renommez-le en « epanet2.dll ». Cette étape peut sembler inutile, mais le .dll ne pourrait pas fonctionner sans ce changement, en raison de la façon dont EPANET a été développé. Pour l'instant, ne faites rien avec, nous allons l'utiliser plus tard.

For the final step, navigate to the EPANET2.3-IMX folder in your file explorer. Then go to the "bin" folder, where you'll find a .dll file named "libepanet2.dll". Rename it to "epanet2.dll". This step might seem unnecessary, but the .dll file wouldn't work without this change, due to the way EPANET was developed. For now, don't do anything with it; we'll use it later.







Compiler le GUI en .exe *Compile the GUI into .exe*

Le GUI (Graphical User Interface) est le programme exécutable (.exe) avec lequel l'utilisateur interagit. Il fait appel au .dll pour exécuter les calculs, puis affiche les résultats à l'écran.

Lorsque vous installez EPANET, celui-ci vient avec un .exe (le GUI) et un .dll (le code qui exécute les calculs). Puisque nous avons modifié le code source du .dll, il faudra aussi compiler le GUI avec celui-ci. Sinon, si vous utilisez le GUI original d'EPANET, les simulations donneront des résultats erronés. De plus, j'ai apporté une modification au code du GUI pour tenir compte des coordonnées des nœuds et jonctions, car le GUI original ignore les coordonnées.

The GUI (Graphical User Interface) is the executable program (.exe) with which the user interacts. It calls the .dll to perform calculations and then displays the results on the screen.

When you install EPANET, it comes with an .exe (the GUI) and a .dll (the code that performs the calculations). Since we modified the source code of the .dll, you will also need to compile the GUI with it. Otherwise, if you use the original EPANET GUI, the simulations will produce incorrect results. Furthermore, I modified the GUI code to account for the coordinates of nodes and junctions, as the original GUI ignores coordinates.

1. Installer Delphi Community Edition *Install Delphi Community Edition*

Pour compiler le GUI en .exe, il faudra installer Delphi Community Edition, car le code est en Pascal. Vous pouvez faire cela en suivant les étapes 1 à 3 du site suivant. Bien que Delphi Community Edition soit gratuit, je vous invite à utiliser votre adresse courriel étudiante pour créer votre compte, par précaution.

Pour l'étape 1, installer seulement « EPANET2.2 Source Code Files ». Le toolkit ne sera pas utilisé.

To compile the GUI into an .exe, you will need to install Delphi Community Edition, as the code is written in Pascal. You can do this by following steps 1 through 3 on the following website. Although Delphi Community Edition is free, I recommend using your student email address to create your account, as a precaution.

For step 1, install only "EPANET2.2 Source Code Files". The toolkit will not be used.

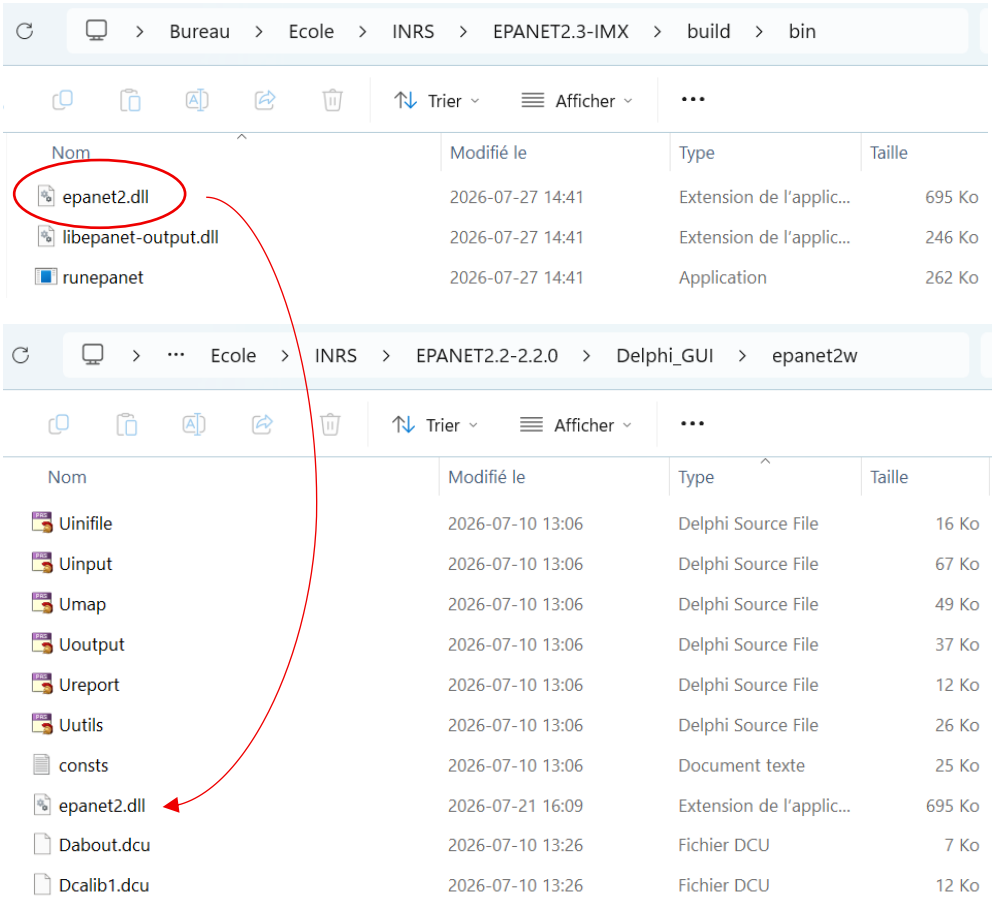
How to Compile the EPANET GUI

Ne fermez pas l'onglet du site après les étapes, nous l'utiliserons prochainement.

2. Remplacer le .dll par le vôtre *Replace the .dll file with your own*

Après avoir terminé les 3 premières étapes, vous serez rendu à la 4e étape, où l'on vous dit de prendre le .dll dans le toolkit et de le mettre dans le dossier Delphi_GUI → epanet2w. Au lieu de cela, prenez le « epanet2.dll » que nous avons compilé à l'étape précédente (qui devrait se trouver dans votre dossier EPANET2.3-IMX → build → bin), et mettez-le dans le dossier Delphi_GUI → epanet2w, comme indiqué sur le site.

After completing the first three steps, you will arrive at step four, where you are instructed to take the .dll file from the toolkit and place it in the Delphi_GUI → epanet2w folder. Instead, take the "epanet2.dll" file that we compiled in the previous step (which should be located in your EPANET2.3-IMX → build → bin folder), and place it in the Delphi_GUI → epanet2w folder, as indicated on the website.

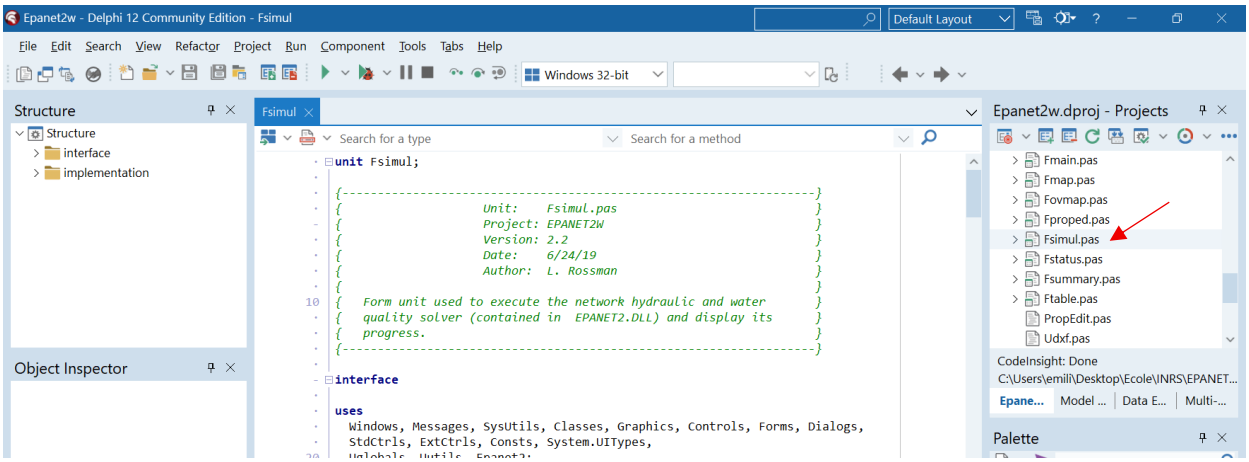


3. Compiler le GUI en .exe *Compile the GUI into an .exe file*

Faites les étapes 5 et 6. Après avoir terminé l'étape 6, ne passez pas directement à l'étape 7, il faudra changer une partie du code.

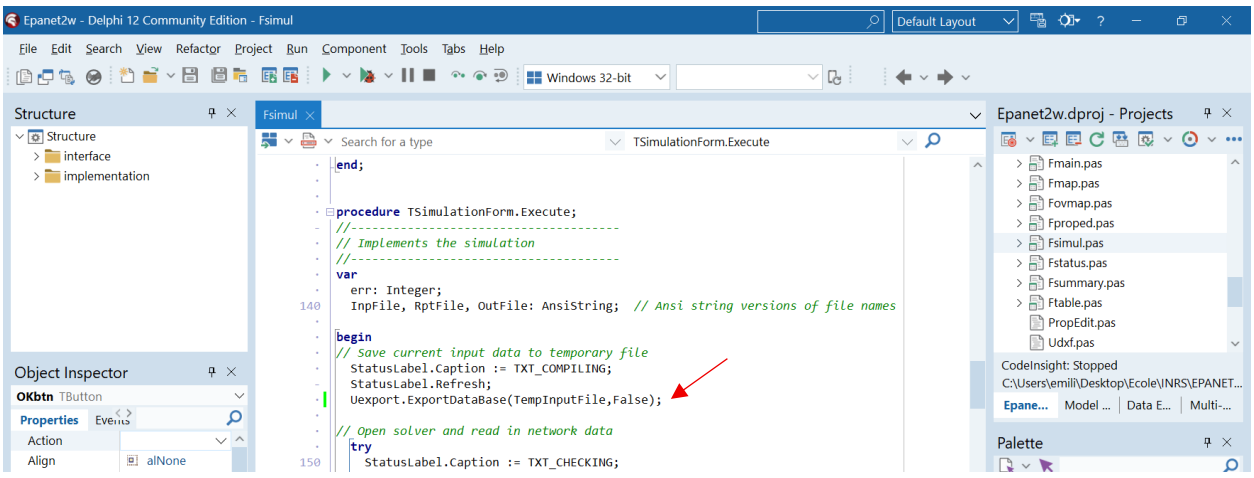
Après avoir sauvegardé les chemins indiqués dans la librairie en cliquant sur « Save », vous devriez vous retrouver sur la page principale où Fmain est ouvert. Complètement à droite, vous devriez retrouver une section Epanet2w.dproj – Projects, faites défiler la liste jusqu'à ce que vous trouviez Fsimul.pas et double-cliquez pour ouvrir le fichier. Le contenu du fichier devrait apparaître à l'écran comme suit :

Complete steps 5 and 6. After completing step 6, do not proceed directly to step 7; you will need to modify some of the code. After saving the specified paths in the library by clicking "Save," you should be taken to the main page where Fmain is open. On the far right, you should find a section called Epanet2w.dproj – Projects. Scroll down until you find Fsimul.pas and double-click it to open the file. The file's contents should appear on the screen as follows:

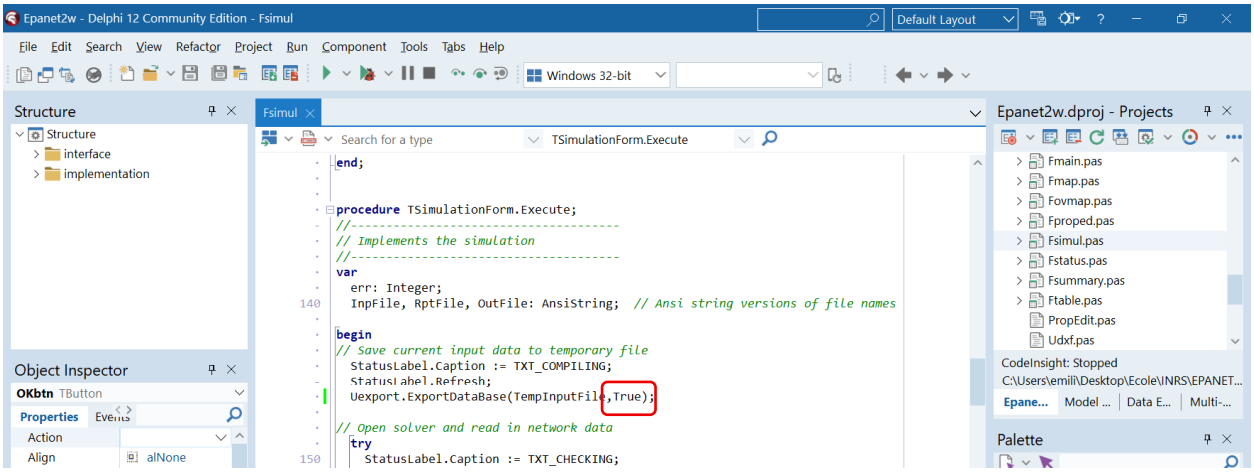


Une fois dans le fichier, descendez jusqu'à la ligne 146, où vous devriez retrouver la ligne suivante (ou faites Ctrl-F) : « Uexport.ExportDataBase(TempInputFile,False); »

Once in the file, scroll down to line 146, where you should find the following line (or do Ctrl-F): "Uexport.ExportDataBase(TempInputFile,False);"



Changez le « False » pour « True » *Change "False" to "True"*



Sauvegardez les modifications, et vous pouvez procéder à l'étape 7 du site internet.

Vous pouvez ensuite tout fermer : l'interface EPANET qui s'ouvre, et Delphi Community Edition.

Save the changes, and you can proceed to step 7 of the website.

You can then close everything: the EPANET interface that opens, and Delphi Community Edition.

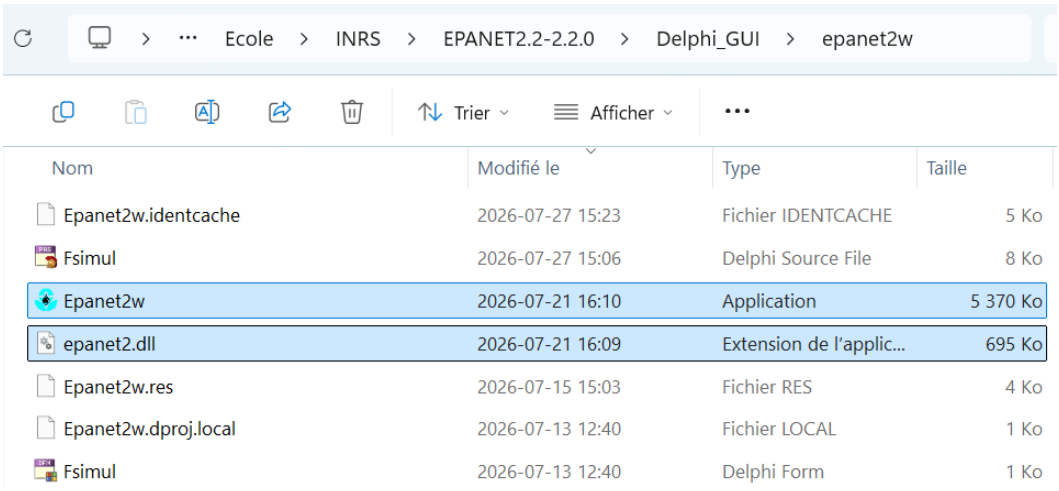
Ouvrir l'application *Open the application*

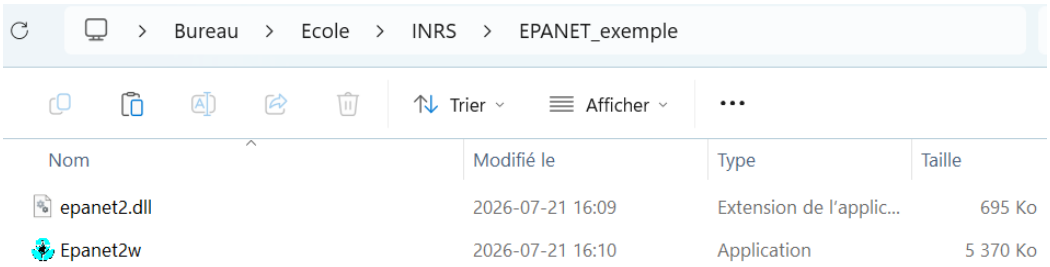
Cette étape sert seulement à simplifier l'accès, l'ouverture et le partage d'EPANET-IMX.

Dans le dossier Delphi_GUI → epanet2w de la section précédente, copiez l'application Epanet2w (celle avec la goutte d'eau) ainsi que epanet2.dll (que vous pouvez aussi retrouver dans le dossier Delphi_GUI → epanet2w). Collez ensuite ces deux fichiers dans un dossier indépendant que vous aurez créé sur votre ordinateur. (Triez par date de modification pour avoir les deux éléments ensemble dans la liste.)

This step is only to simplify accessing, opening, and sharing EPANET-IMX.

In the Delphi_GUI → epanet2w folder from the previous section, copy the Epanet2w application (the one with the water droplet icon) and epanet2.dll (which you can also find in the Delphi_GUI → epanet2w folder). Then paste these two files into a separate folder that you have created on your computer. (Sort by modification date to have both items together in the list.)





Pour ouvrir l'application et l'utiliser, double-cliquez sur Epanet2w (ou ouvrez-la par clic droit).

Pour partager l'application, compressez le dossier que vous avez créé : certaines plateformes de communication comme Outlook ou Teams bloquent l'envoi du fichier seul (ou du .dll seul) pour des raisons de sécurité.

To open and use the application, double-click on Epanet2w (or right-click on it).

To share the application, compress the folder you created: some communication platforms like Outlook or Teams block sending the file alone (or the .dll file alone) for security reasons.